

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3029 - KHAI PHÁ DỮ LIỆU

Báo cáo cuối kì

Ứng dụng Data Mining trong dự đoán giá nhà

Advisor(s): Đỗ Thanh Thái

Student(s): Võ Ngọc Tú ID 2213857

Trần Tuấn Kiệt ID 2311787

Bùi Hà Hải ID 2310862

Link GitHub: [Data-Mining-252](#)

HO CHI MINH CITY, APRIL 2026



Phân công công việc

No.	Họ và Tên	MSSV	Công việc	Phần trăm
1	Trần Tuấn Kiệt	2311787	Tiền xử lý và phân tích dữ liệu Đánh giá và so sánh các mô hình	100%
2	Võ Ngọc Tú	2213857	MLP Neural network XGBoost Regression	100%
3	Bùi Hà Hải	2310862	Random Forest Regression Linear Regression	100%



Contents

1	Tiền xử lý dữ liệu	8
1.1	Tổng quan về dữ liệu	8
1.2	Xử lý dữ liệu	9
1.2.1	Đọc dữ liệu	9
1.2.2	Kiểm tra và xử lý dữ liệu khuyết	9
1.2.3	Xử lý dữ liệu phân loại	9
2	Phân tích dữ liệu	11
3	Kiểm tra phân bố và outliers của "price"	12
3.1	Ma trận tương quan của các biến	13
3.2	Trực quan hóa dữ liệu	15
3.2.1	Biến <code>area</code>	15
3.2.2	Biến <code>"bedrooms"</code>	16
3.2.3	Biến <code>"bathrooms"</code>	16
3.2.4	Biến <code>"stories"</code>	17
3.2.5	Biến <code>"parking"</code>	17
3.2.6	Biến <code>"mainroad"</code>	18
3.2.7	Biến <code>"guestroom"</code>	18
3.2.8	Biến <code>"basement"</code>	19
3.2.9	Biến <code>"hotwaterheating"</code>	19



3.2.10	Biến "airconditioning"	20
3.2.11	Biến "prefarea"	20
3.2.12	Biến "furnishingstatus_semi-furnished"	21
3.2.13	Biến "furnishingstatus_unfurnished"	21
4	Phương pháp	22
4.1	Hồi quy tuyến tính	22
4.2	Random Forest	24
4.3	XGBoost	26
4.4	GridSearchCV	29
4.5	Mạng nơ-ron nhiều lớp	30
5	Xây dựng mô hình dự đoán	33
5.1	Hồi quy tuyến tính	34
5.2	Random Forest	35
5.3	XGBoost	37
5.4	Mạng Nơ-ron nhiều lớp	38
6	Đánh giá mô hình dự đoán	40
6.1	Các Phương pháp Đánh giá Hiệu quả Mô hình	40
6.2	Kết quả đánh giá	41
6.3	Đồ thị đánh giá	42
6.4	Kết luận	44



7 Tổng Kết

45

List of Figures

1.1	Thông tin cơ bản về tập dữ liệu	9
1.2	Thống kê dữ liệu khuyết và trùng lặp	10
1.3	Thống kê kiểu dữ liệu	10
1.4	Thống kê kiểu dữ liệu sau khi phân loại	11
2.1	Tập dữ liệu sau khi xử lý	11
3.1	Giá trị outliers của "price"	12
3.2	Biểu đồ phân bố "price"	13
3.3	Biểu đồ tương quan giữa các biến	14
3.4	Biến "area"	15
3.5	Biến "bedrooms"	16
3.6	Biến "bathrooms"	16
3.7	Biến "stories"	17
3.8	Biến "parking"	17
3.9	Biến "mainroad"	18
3.10	Biến "guestroom"	18
3.11	Biến "basement"	19
3.12	Biến "hotwaterheating"	19
3.13	Biến "airconditioning"	20



3.14	Biến "prefarea"	20
3.15	Biến "furnishingstatus_semi-furnished"	21
3.16	Biến "furnishingstatus_unfurnished"	21
4.1	Mạng nơ-ron nhân tạo nhiều lớp	31
5.1	Chia tập dữ liệu Train/Test	33
5.2	Chuẩn hóa dữ liệu với StandardScaler	34
5.3	Mô hình hồi quy tuyến tính	34
5.4	Tối ưu tham số với GridSearchCV	36
5.5	Bộ tham số tối ưu	36
5.6	Cấu hình không gian tìm kiếm GridSearchCV cho XGBoost	38
5.7	Bộ tham số tốt nhất cho mô hình	38
5.8	Phương pháp mạng Nơ-ron nhiều lớp	39
5.9	Đồ thị mất mát	40
6.1	Linear Regression: (a) Predictions vs Actual, (b) Residuals vs Predictions, (c) Distribution of Residuals	42
6.2	Random Forest: (a) Predictions vs Actual, (b) Residuals vs Predictions, (c) Distribution of Residuals	42
6.3	XGBoost	43
6.4	Mạng Nơ-ron nhiều lớp	43

List of Tables

6.1	Các chỉ số đánh giá hiệu suất mô hình	41
-----	---	----



1 Tiền xử lý dữ liệu

1.1 Tổng quan về dữ liệu

File dữ liệu đầu vào được nhóm quyết định chọn trên nền tảng Kaggle có tên **Housing.csv** tại [housing-prices-dataset](#). Bộ dữ liệu gồm các thông tin như giá bất động sản, diện tích, số phòng ngủ, số phòng tắm, v.v. Mục tiêu của bài tập lớn này là tìm ra mối quan hệ giữa giá bất động sản và các đặc tính của căn nhà.

Bộ dữ liệu bao gồm 545 mẫu, mỗi mẫu đại diện cho một căn nhà với các cột đặc tính sau:

- **price**: giá bán của căn nhà (đơn vị: USD)
- **area**: diện tích của căn nhà (feet vuông)
- **bedrooms**: số lượng phòng ngủ
- **bathrooms**: số lượng phòng tắm
- **mainroad**: có nằm ở mặt tiền hay không
- **guestroom**: có phòng khách hay không
- **basement**: có tầng hầm hay không
- **hotwaterheating**: có máy nước nóng hay không
- **airconditioning**: có điều hòa hay không
- **parking**: số lượng bãi đậu xe
- **prefarea**: thuộc khu vực ưu tiên hay không
- **furnishingstatus**: tình trạng nội thất (unfurnished, semi-furnished, furnished)

1.2 Xử lý dữ liệu

1.2.1 Đọc dữ liệu

Sau khi đọc dữ liệu, ta lưu vào Pandas DataFrame.

Hình bên dưới minh họa 5 dòng đầu trong bộ dữ liệu để có cái nhìn tổng quát hơn.

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished

Figure 1.1: Thông tin cơ bản về tập dữ liệu

1.2.2 Kiểm tra và xử lý dữ liệu khuyết

Để kiểm tra dữ liệu khuyết và dữ liệu trùng trong tập dữ liệu, ta sử dụng 2 lệnh: `data.isnull().sum()` và `data.duplicated().sum()` để thống kê lại tổng số dữ liệu khuyết và dữ liệu trùng lặp trên từng cột:

Dựa vào hình trên, ta có thể thấy rằng không có giá trị khuyết và giá trị trùng lặp tồn tại trong tập dữ liệu. Do đó, không cần xử lý dữ liệu khuyết cũng như dữ liệu trùng lặp.

1.2.3 Xử lý dữ liệu phân loại

Để kiểm tra sự tồn tại của dữ liệu phân loại trong tập dữ liệu, ta sử dụng lệnh `data.dtypes` nhằm thống kê các kiểu dữ liệu của các đặc trưng.

Missing Values	
price	0
area	0
bedrooms	0
bathrooms	0
stories	0
mainroad	0
guestroom	0
basement	0
hotwaterheating	0
airconditioning	0
parking	0
prefarea	0
furnishingstatus	0
Duplicate values:	
0	

Figure 1.2: Thống kê dữ liệu khuyết và trùng lặp

Data Type	
price	int64
area	int64
bedrooms	int64
bathrooms	int64
stories	int64
mainroad	object
guestroom	object
basement	object
hotwaterheating	object
airconditioning	object
parking	int64
prefarea	object
furnishingstatus	object

Figure 1.3: Thống kê kiểu dữ liệu

Dựa vào hình, ta có thể thấy rằng các cột có kiểu dữ liệu là object gồm: `mainroad`, `guestroom`, `basement`, `hotwaterheating`, `airconditioning`, `prefarea`, và `furnishingstatus` – đây là các cột chứa giá trị phân loại. Ta áp dụng kỹ thuật One-Hot Encoding để chuyển đổi các cột này thành các giá trị nhị phân tương ứng với từng loại giá trị. Ngoài ra, ta sẽ sử dụng tham số `drop_first=True` để tránh trường hợp đa cộng tuyến. Sau khi xử lý, tập dữ liệu sẽ được chuyển hóa thành như sau:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus_semi-furnished	furnishingstatus_unfurnished
0	13300000	7420	4	2	3	1	0	0	0	1	2	1	0	0
1	12250000	8960	4	4	4	1	0	0	0	1	3	0	0	0
2	12250000	9960	3	2	2	1	0	1	0	0	2	1	1	0
3	12215000	7500	4	2	2	1	0	1	0	1	3	1	0	0
4	11410000	7420	4	1	2	1	1	1	0	1	2	0	0	0

Figure 1.4: Thống kê kiểu dữ liệu sau khi phân loại

2 Phân tích dữ liệu

Data columns (total 14 columns):			
#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	price	545 non-null	int64
1	area	545 non-null	int64
2	bedrooms	545 non-null	int64
3	bathrooms	545 non-null	int64
4	stories	545 non-null	int64
5	mainroad	545 non-null	int64
6	guestroom	545 non-null	int64
7	basement	545 non-null	int64
8	hotwaterheating	545 non-null	int64
9	airconditioning	545 non-null	int64
10	parking	545 non-null	int64
11	prefarea	545 non-null	int64
12	furnishingstatus_semi-furnished	545 non-null	int32
13	furnishingstatus_unfurnished	545 non-null	int32
dtypes: int32(2), int64(12)			

Figure 2.1: Tập dữ liệu sau khi xử lý

Trước khi phân tích dữ liệu, ta phân tích tổng quan dữ liệu sau tiền xử lý. Ta sử dụng `df.info()`. Dựa vào Hình 3.1, ta có thể rút ra được rằng:

- Dữ liệu bao gồm 545 mẫu.
- Ở mỗi cột dữ liệu đều đếm được 545 dòng dữ liệu không rỗng.
- Tất cả 14 cột đều có kiểu `int64`.

Trong đó, biến dữ liệu được chia thành hai loại: biến liên tục và biến rời rạc.

- Biến liên tục: là biến có thể nhận vô số giá trị $\rightarrow$ `area`.
- Biến rời rạc: là biến chỉ có thể nhận hữu hạn các giá trị riêng biệt $\rightarrow$ `bedrooms`, `bathrooms`, `stories`, `parking`, `mainroad_yes`, `guestroom_yes`, `basement_yes`, `hotwaterheating_yes`, `airconditioning_yes`, `prefarea_yes`, `furnishingstatus_semi-furnished`, và `furnishingstatus_furnished`.

3 Kiểm tra phân bố và outliers của "price"

Trước khi phân tích sự tương quan của các thuộc tính, ta cần kiểm tra sự phân bố của "price":

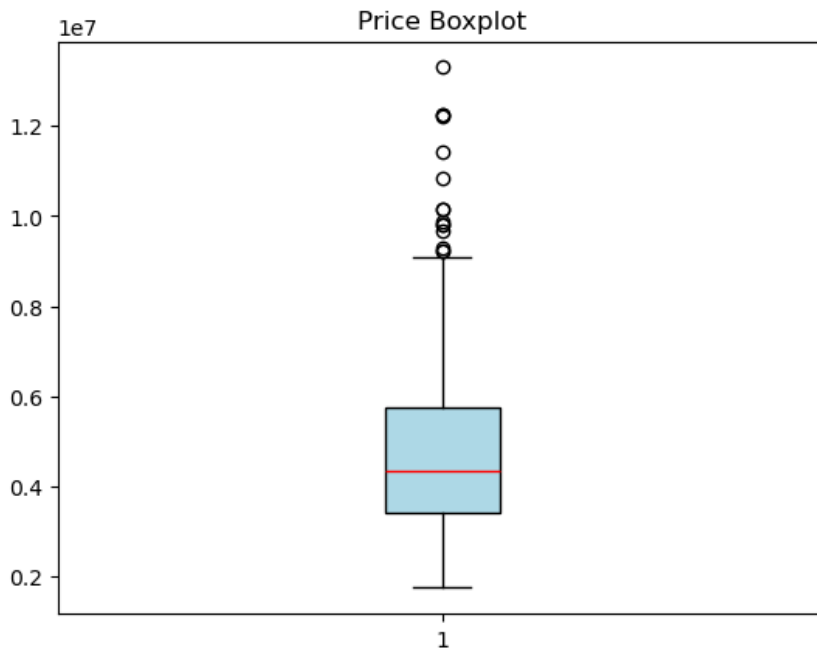


Figure 3.1: Giá trị outliers của "price"

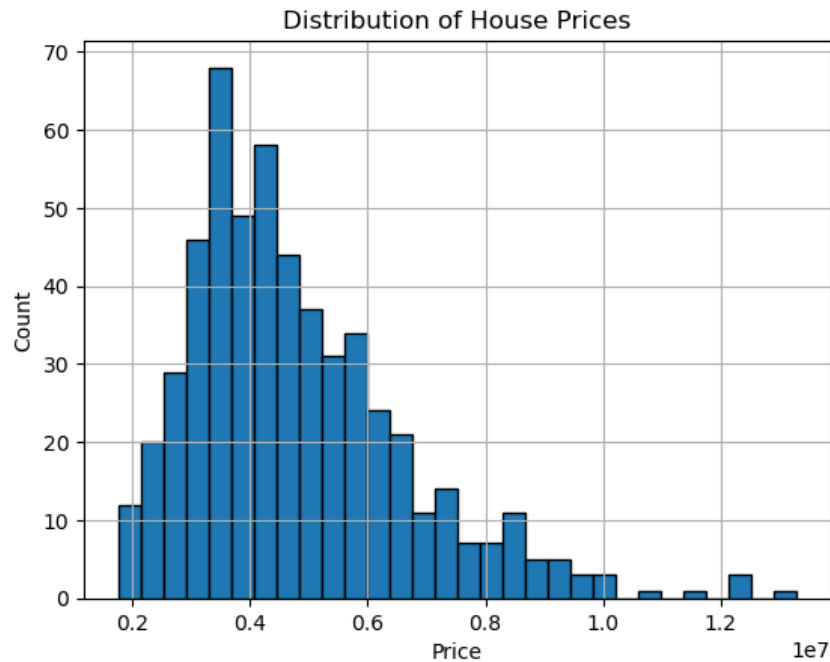


Figure 3.2: Biểu đồ phân bố "price"

Đánh giá chung: Biến `price` có phân phối lệch phải (right-skewed) rõ rệt, với phần lớn dữ liệu tập trung ở phân khúc giá thấp và trung bình (khoảng 3×10^6 đến 6×10^6). Cả hai biểu đồ đều xác nhận sự hiện diện của nhiều giá trị ngoại lệ (outliers) phía cao, cho thấy thị trường tồn tại một số bất động sản có giá trị đột biến so với mặt bằng chung. Đặc điểm này gợi ý rằng việc áp dụng biến đổi logarit có thể cần thiết để đưa dữ liệu về phân phối chuẩn hơn trước khi đưa vào các mô hình dự báo.

3.1 Ma trận tương quan của các biến

Để phân tích mối quan hệ giữa các đặc trưng trong tập dữ liệu, ta sử dụng ma trận tương quan. Ma trận này biểu diễn mức độ tương quan tuyến tính giữa các cặp thuộc tính, với giá trị nằm trong khoảng từ -1 đến 1 . Trong đó:

- Giá trị gần 1 biểu thị mối tương quan thuận mạnh.
- Giá trị gần -1 biểu thị mối tương quan nghịch mạnh.
- Giá trị gần 0 biểu thị mối tương quan yếu hoặc không tồn tại.

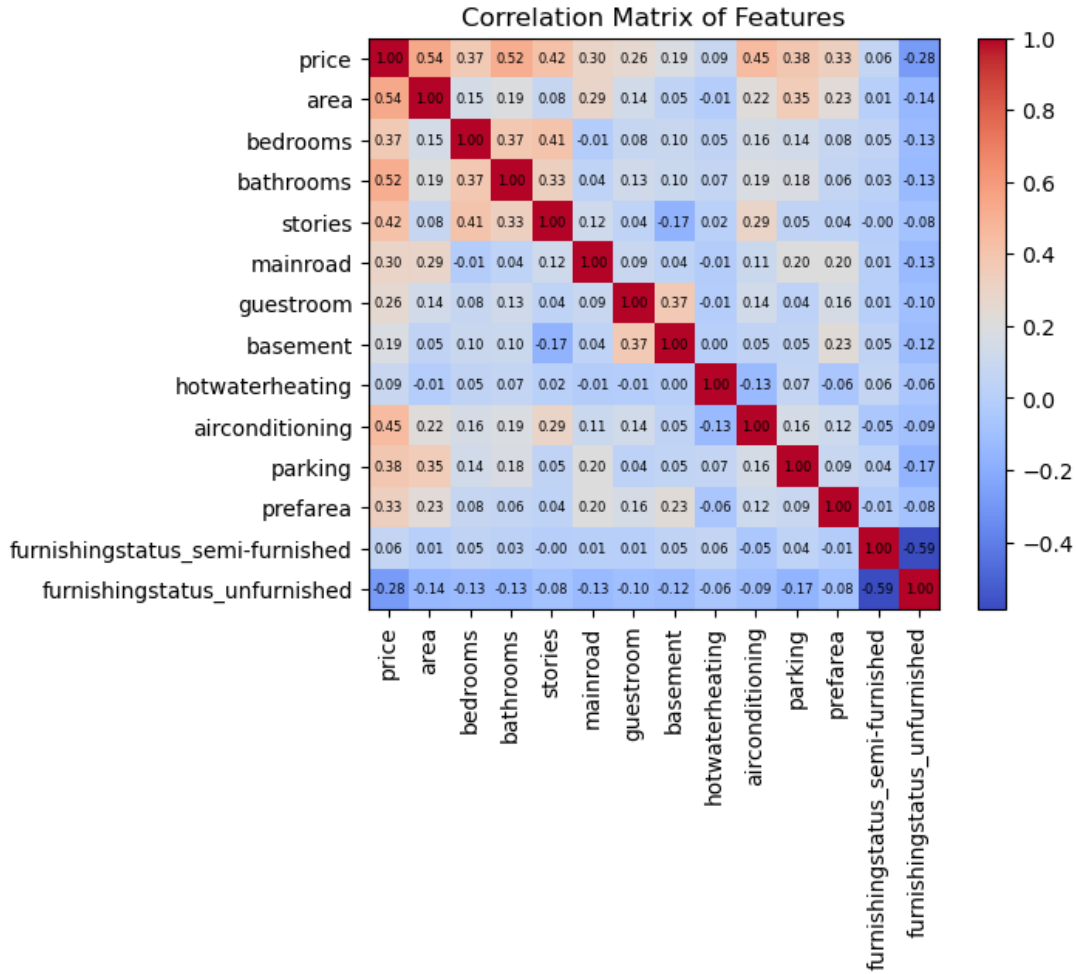


Figure 3.3: Biểu đồ tương quan giữa các biến

Dựa vào hình, ta nhận thấy một số biến có mối tương quan đáng kể với biến mục tiêu **price**. Cụ thể, các biến như **area** ($r = 0.54$), **bathrooms** ($r = 0.52$), **stories** ($r = 0.45$), và **airconditioning** ($r = 0.42$) có mức tương quan khá mạnh, cho thấy chúng có ảnh hưởng rõ rệt đến giá nhà.

Bên cạnh đó, các biến như **bedrooms**, **parking**, **mainroad** và **prefarea** có hệ số tương quan ở mức trung bình ($0.30 \leq r < 0.4$), vẫn có thể được xem xét trong quá trình xây dựng mô hình.

Ngược lại, các đặc trưng như **basement**, **guestroom**, **hotwaterheating** và **furnishingstatus** có tương quan yếu hoặc gần như không có tương quan với giá nhà ($r < 0.2$), thậm chí có

một số biến như `mainroad_no` thể hiện mối quan hệ ngược chiều ($r = -0.28$).

3.2 Trực quan hóa dữ liệu

Trực quan hóa dữ liệu sẽ giúp ta hiểu rõ hơn về phân bố, xu hướng và mối quan hệ giữa các biến và biến `price`.

- Biến liên tục: ta sử dụng boxplot để biểu thị sự phân bố của nó và scatterplot để

biểu hiện phân bố theo `price`

- Biến rời rạc: ta sử dụng barplot để biểu thị sự phân bố của nó và boxplot để biểu hiện phân bố theo `price`

3.2.1 Biến `area`

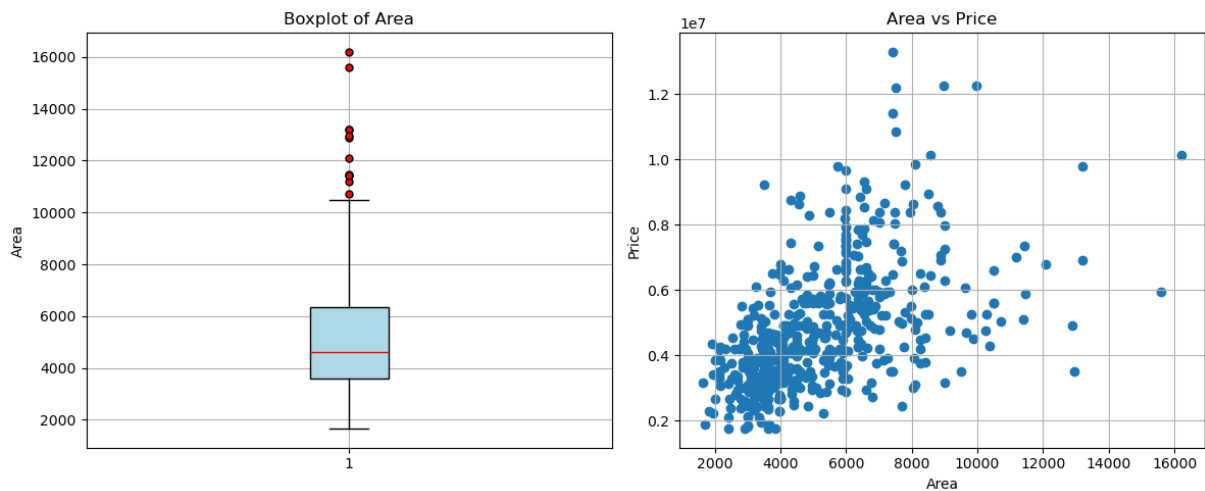


Figure 3.4: Biến "area"

3.2.2 Biến "bedrooms"

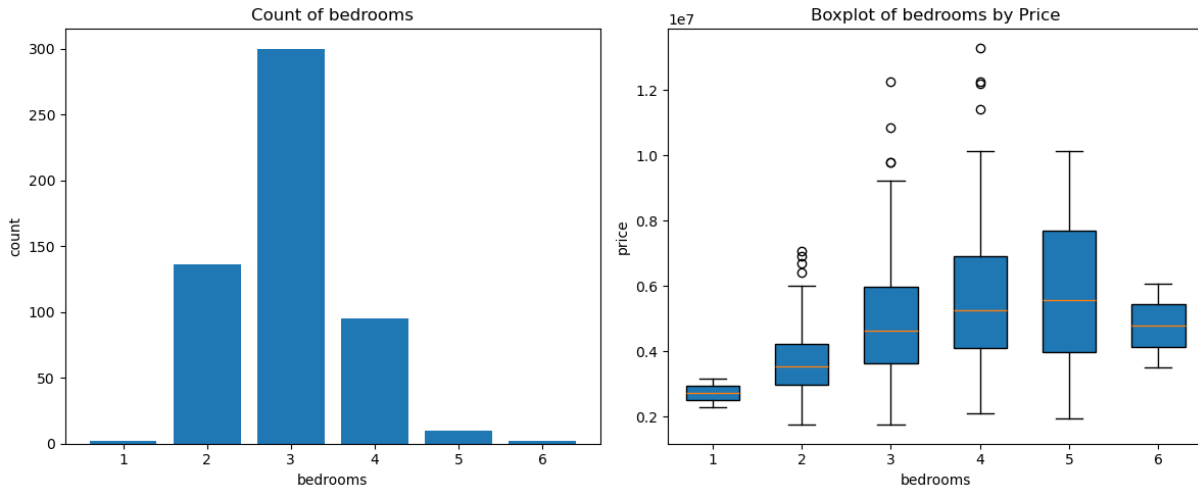


Figure 3.5: Biến "bedrooms"

3.2.3 Biến "bathrooms"

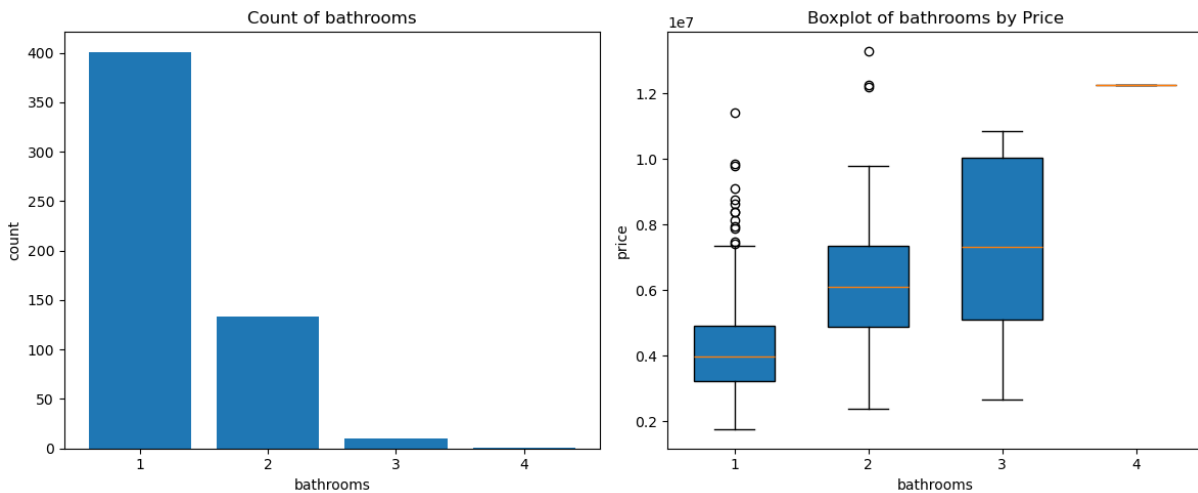


Figure 3.6: Biến "bathrooms"

3.2.4 Biến "stories"

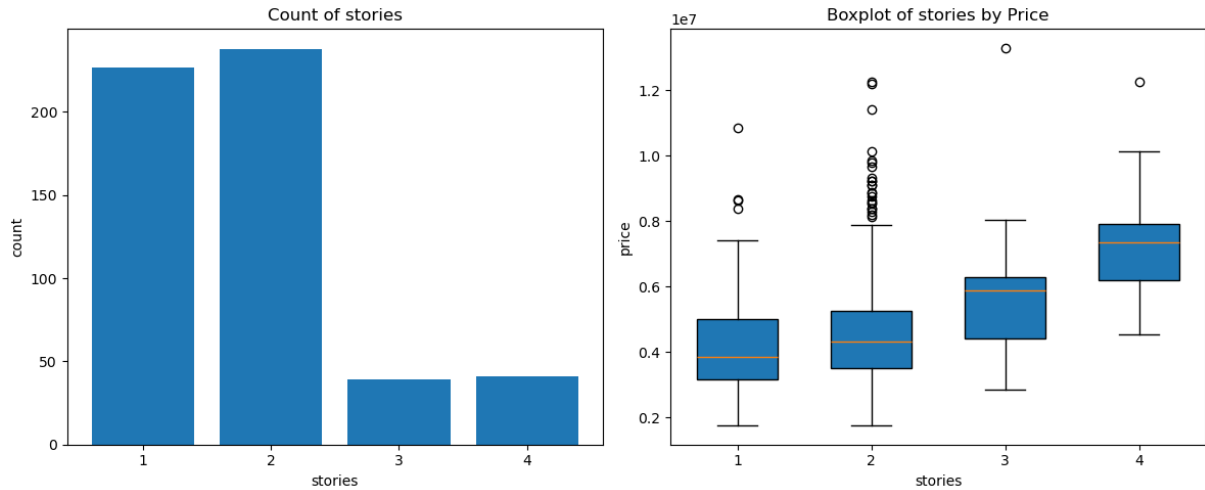


Figure 3.7: Biến "stories"

3.2.5 Biến "parking"

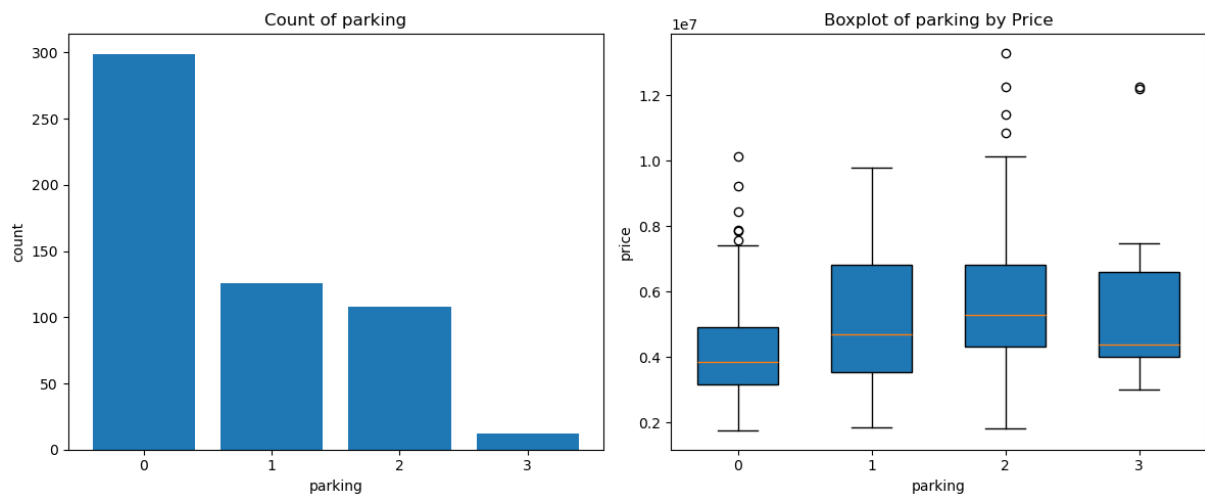


Figure 3.8: Biến "parking"

3.2.6 Biến "mainroad"

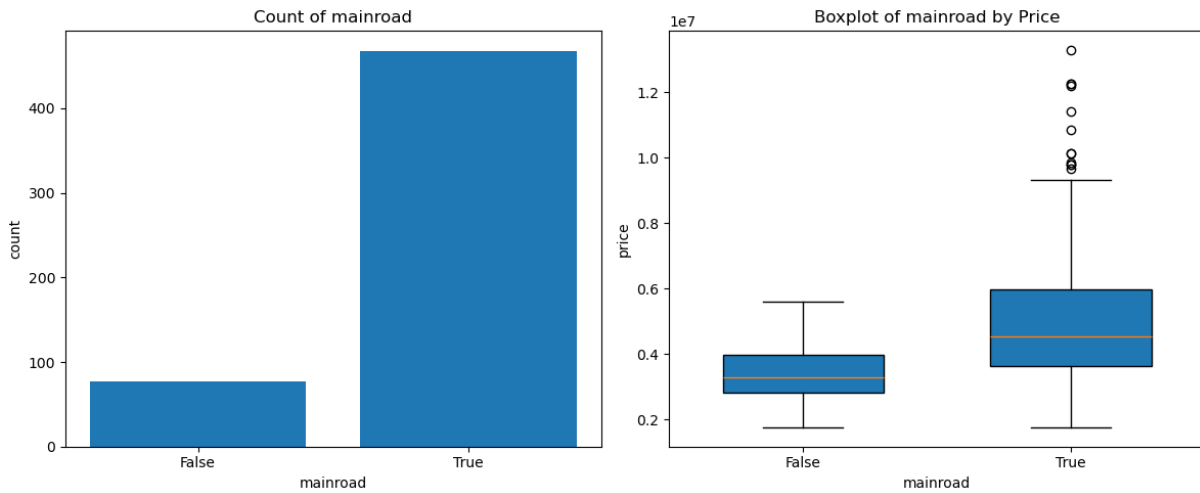


Figure 3.9: Biến "mainroad"

3.2.7 Biến "guestroom"

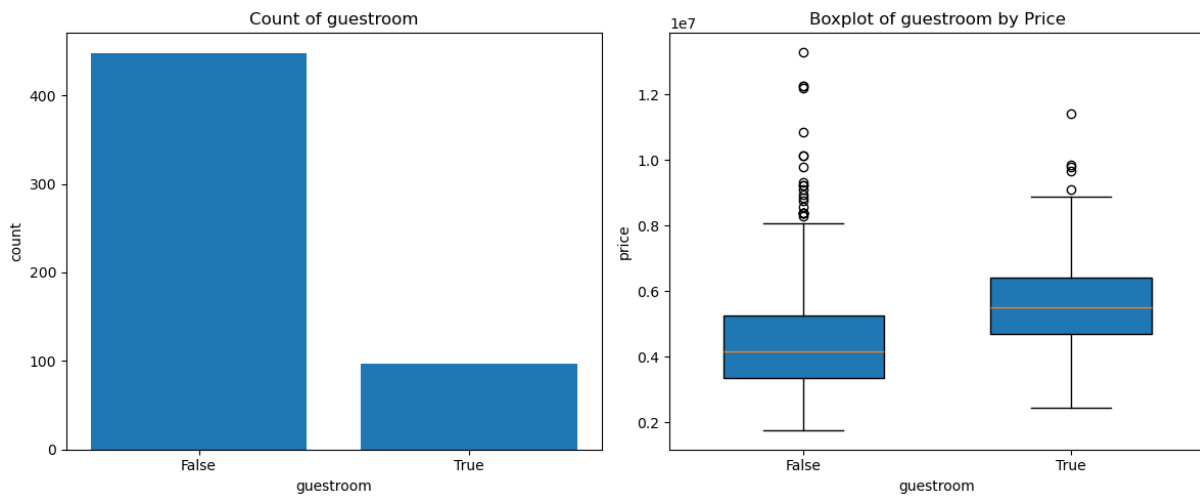


Figure 3.10: Biến "guestroom"

3.2.8 Biến "basement"

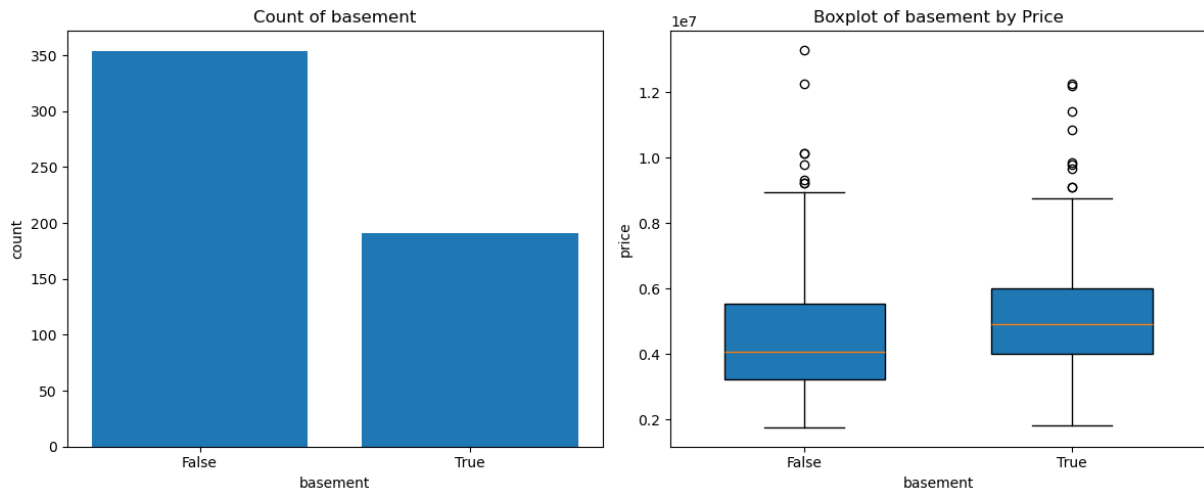


Figure 3.11: Biến "basement"

3.2.9 Biến "hotwaterheating"

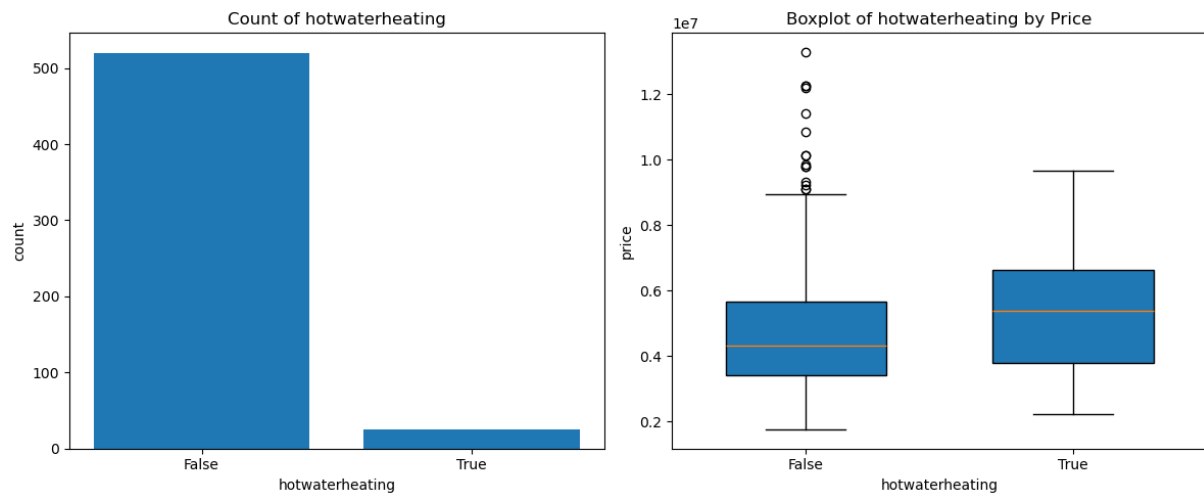


Figure 3.12: Biến "hotwaterheating"

3.2.10 Biến "airconditioning"

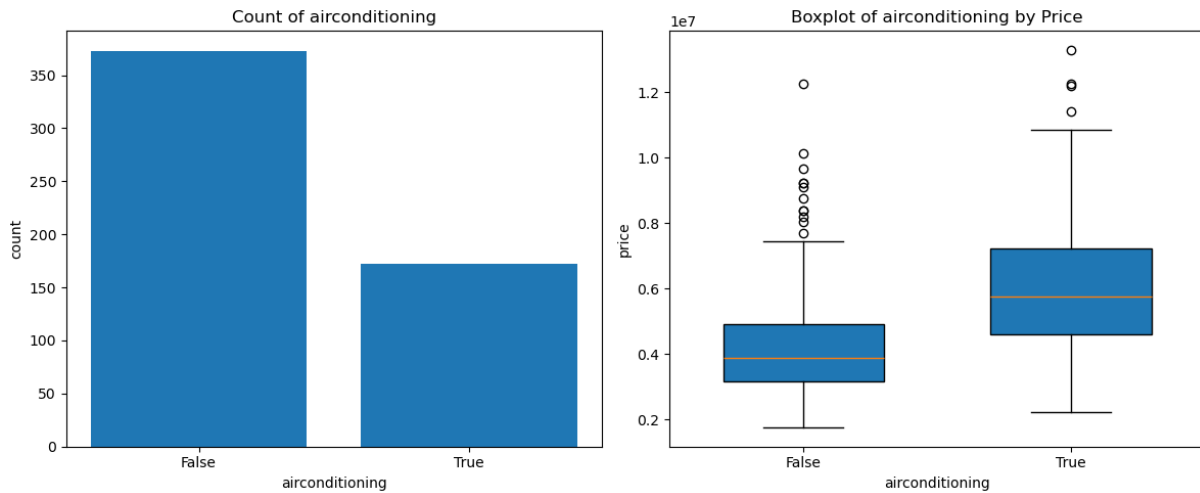


Figure 3.13: Biến "airconditioning"

3.2.11 Biến "prefarea"

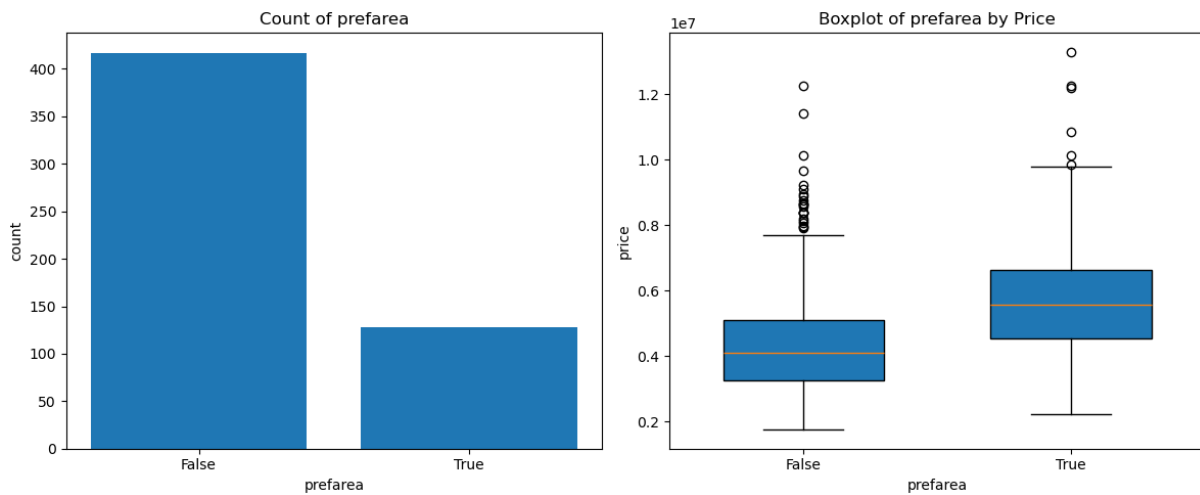


Figure 3.14: Biến "prefarea"

3.2.12 Biến "furnishingstatus_semi-furnished"

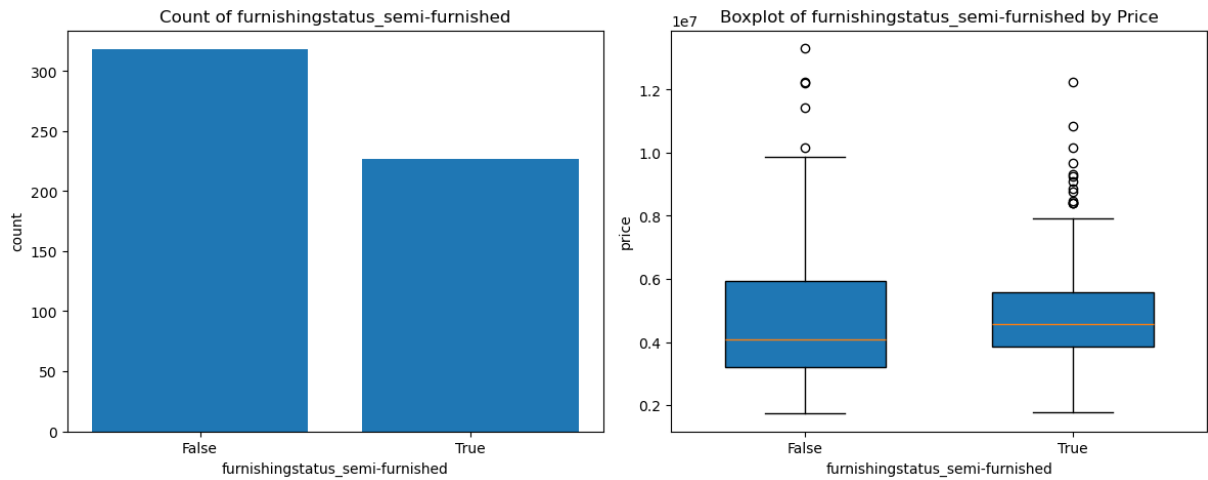


Figure 3.15: Biến "furnishingstatus_semi-furnished"

3.2.13 Biến "furnishingstatus_unfurnished"

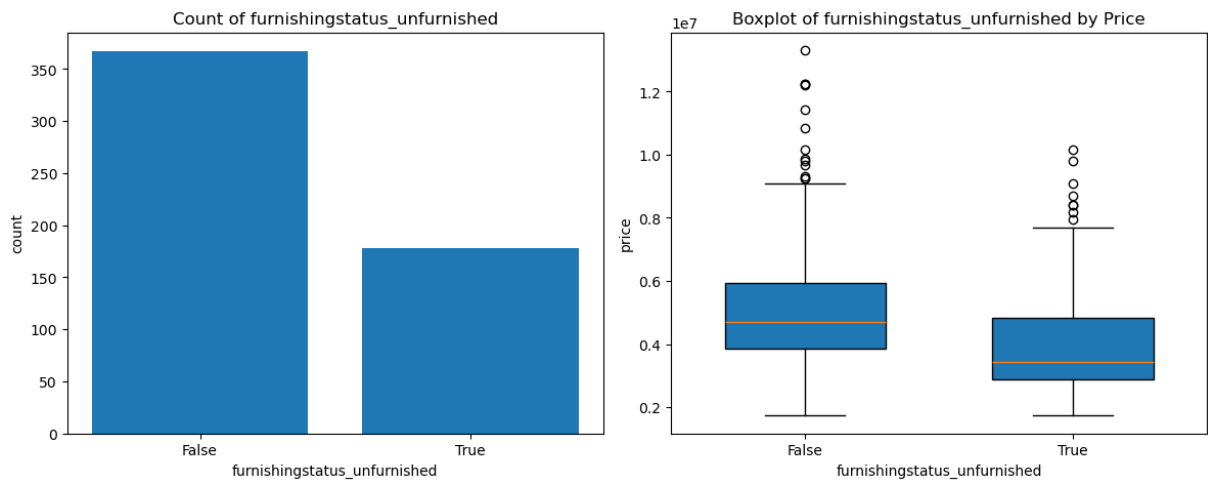


Figure 3.16: Biến "furnishingstatus_unfurnished"

4 Phương pháp

4.1 Hồi quy tuyến tính

Hồi quy tuyến tính (Linear Regression) là một trong những phương pháp cơ bản và phổ biến nhất trong *Data Mining* và *Machine Learning*, được sử dụng để mô hình hóa mối quan hệ giữa một biến phụ thuộc và một hoặc nhiều biến độc lập. Mục tiêu của mô hình là tìm ra một hàm tuyến tính tốt nhất để dự đoán giá trị đầu ra dựa trên dữ liệu đầu vào.

Giả định cốt lõi của hồi quy tuyến tính là tồn tại mối quan hệ tuyến tính giữa các biến, tức là đầu ra có thể được biểu diễn dưới dạng tổ hợp tuyến tính của các biến đầu vào.

Nguyên lý hoạt động:

- **Hồi quy tuyến tính đơn (một biến đầu vào):**

$$y = wx + b$$

Trong đó:

- y : biến phụ thuộc (giá trị cần dự đoán)
- x : biến độc lập
- w : hệ số hồi quy (trọng số)
- b : hệ số lệch (bias/intercept)

- **Hồi quy tuyến tính đa biến:**

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T \mathbf{x} + b$$

Với:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T, \quad \mathbf{w} = [w_1, w_2, \dots, w_n]^T$$

Để đánh giá chất lượng mô hình, hàm mất mát phổ biến được sử dụng là **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

Trong đó:

- m : số lượng mẫu
- $y^{(i)}$: giá trị thực tế
- $\hat{y}^{(i)}$: giá trị dự đoán

Mục tiêu của quá trình huấn luyện là tìm bộ tham số $\mathbf{w}, b$ sao cho MSE được tối thiểu hóa. Điều này thường được thực hiện thông qua các phương pháp tối ưu như *Gradient Descent* hoặc nghiệm giải tích (*Normal Equation*).

Ưu điểm và hạn chế:

- **Ưu điểm:**
 - Đơn giản, dễ triển khai và huấn luyện nhanh
 - Dễ diễn giải, cho phép hiểu rõ ảnh hưởng của từng đặc trưng
 - Hoạt động tốt với dữ liệu có quan hệ tuyến tính
- **Hạn chế:**
 - Không mô hình hóa được quan hệ phi tuyến
 - Nhạy cảm với outliers
 - Có thể bị underfitting nếu dữ liệu phức tạp

Trong bài toán dự đoán giá nhà, hồi quy tuyến tính là một mô hình cơ sở (*baseline*) hiệu quả, giúp mô tả mối quan hệ giữa các đặc trưng như diện tích, số phòng, vị trí và giá trị bất động sản. Mặc dù đơn giản, mô hình vẫn cung cấp khả năng dự đoán tốt và đặc biệt hữu ích trong việc phân tích mức độ ảnh hưởng của từng yếu tố.

4.2 Random Forest

Random Forest là một phương pháp học máy thuộc nhóm *Ensemble Learning*, được đề xuất nhằm cải thiện độ chính xác và khả năng tổng quát hóa của mô hình bằng cách kết hợp nhiều cây quyết định (*Decision Trees*). Phương pháp này có thể áp dụng cho cả bài toán phân loại và hồi quy.

Trong bài toán hồi quy (Random Forest Regression), mô hình dự đoán giá trị liên tục bằng cách lấy trung bình kết quả từ nhiều cây hồi quy.

Nguyên lý hoạt động:

Giả sử có T cây hồi quy $f_1(x), f_2(x), \dots, f_T(x)$. Mỗi cây được huấn luyện độc lập trên một tập dữ liệu con được tạo ra bằng phương pháp **Bootstrap Sampling** từ tập dữ liệu gốc D :

$$D_t \sim \text{Bootstrap}(D), \quad t = 1, 2, \dots, T$$

Ngoài ra, tại mỗi nút của cây, chỉ một tập con ngẫu nhiên các đặc trưng $\mathcal{F}_t \subseteq \mathcal{F}$ được sử dụng để tìm điểm chia tối ưu. Điều này giúp tăng tính đa dạng giữa các cây và giảm tương quan giữa chúng.

Việc phân chia dữ liệu tại mỗi nút dựa trên tiêu chí giảm thiểu sai số, thường sử dụng **Sum of Squared Errors (SSE)**:

$$\text{SSE} = \sum_{i \in S} (y^{(i)} - \bar{y}_S)^2$$

Trong đó:

$$\bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y^{(i)}$$

Cây sẽ tiếp tục phát triển cho đến khi đạt các điều kiện dừng như:

- Độ sâu tối đa

- Số mẫu tối thiểu tại một nút
- Mức giảm sai số không còn đáng kể

Sau khi huấn luyện, dự đoán cuối cùng của mô hình được tính bằng trung bình các dự đoán từ tất cả các cây:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x)$$

- T : số lượng cây
- $f_t(x)$: dự đoán của cây thứ t
- $\hat{y}$: giá trị dự đoán cuối cùng

Ưu điểm và hạn chế:

- **Ưu điểm:**
 - Mô hình hóa tốt các quan hệ phi tuyến phức tạp
 - Giảm overfitting nhờ cơ chế ensemble
 - Ít nhạy cảm với nhiễu và outliers
 - Tự động đánh giá mức độ quan trọng của đặc trưng
- **Hạn chế:**
 - Chi phí tính toán cao hơn hồi quy tuyến tính
 - Khó diễn giải hơn (black-box hơn)
 - Mô hình lớn, tốn bộ nhớ

Trong bài toán dự đoán giá nhà, Random Forest cho phép mô hình hóa các mối quan hệ phi tuyến giữa các yếu tố như diện tích, vị trí địa lý, tiện ích xung quanh và giá trị bất động sản. Nhờ khả năng tổng hợp nhiều cây quyết định, mô hình mang lại độ chính xác cao và khả năng tổng quát hóa tốt hơn so với các phương pháp tuyến tính.

4.3 XGBoost

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy mạnh mẽ thuộc nhóm ensemble learning, cụ thể là phương pháp boosting. Nó được thiết kế để tối ưu hiệu năng và tốc độ khi huấn luyện mô hình cây quyết định.

Nguyên lý hoạt động:

XGBoost xây dựng mô hình bằng cách kết hợp nhiều cây quyết định theo từng vòng lặp. Mỗi cây mới được huấn luyện nhằm sửa lỗi của cây trước đó, giúp mô hình học tốt hơn theo thời gian.

1. Hàm mục tiêu

Ở vòng boosting thứ t , mô hình XGBoost tối ưu hàm mục tiêu sau:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Trong đó:

- l : hàm mất mát (thường dùng MSE: $l(y, \hat{y}) = (y - \hat{y})^2$)
- $\hat{y}_i^{(t-1)}$: dự đoán của mô hình tại bước $t - 1$
- f_t : cây quyết định mới tại bước t
- $\Omega(f_t)$: thành phần regularization giúp chống overfitting

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Trong đó:

- T : số lá của cây

- w_j : trọng số lá thứ j
- γ, λ : hệ số điều chỉnh độ phức tạp mô hình

2. Xấp xỉ Taylor bậc hai

XGBoost sử dụng xấp xỉ Taylor bậc hai để làm tròn hàm mất mát:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Trong đó: - $g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$ (đạo hàm bậc 1) - $h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$ (đạo hàm bậc 2)

3. Tính giá trị tối ưu tại mỗi lá

Với mỗi lá j , tập mẫu thuộc lá là I_j . Tổng gradient và hessian tại lá:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Giá trị đầu ra tối ưu tại lá:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Giá trị hàm mục tiêu tại điểm cực tiểu:

$$\mathcal{L}^{(t)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

Quá trình huấn luyện:

Quá trình huấn luyện XGBoost bao gồm các bước chính sau:

1. **Khởi tạo mô hình:** Dự đoán ban đầu là giá trị trung bình (cho hồi quy) hoặc xác suất (cho phân loại).
2. **Tính toán gradient và hessian:** Các gradient (g_i) và hessian (h_i) được tính cho mỗi mẫu dựa trên hàm mất mát.
3. **Xây dựng cây quyết định:** Mỗi cây mới được xây dựng để tối ưu hóa hàm mất mát với việc sử dụng gradient và hessian.
4. **Cập nhật dự đoán:** Dự đoán của mô hình được cập nhật bằng cách cộng thêm giá trị từ cây mới.
5. **Regularization:** Thành phần regularization giúp kiểm soát độ phức tạp của mô hình và ngăn ngừa overfitting.
6. **Lặp lại quá trình:** Các bước trên được lặp lại cho đến khi đạt số vòng lặp tối đa hoặc không còn cải thiện đáng kể.

XGBoost là một lựa chọn mạnh mẽ cho bài toán hồi quy dự đoán giá nhà nhờ vào khả năng xử lý các mối quan hệ phi tuyến tính phức tạp trong dữ liệu. Mô hình này sử dụng kỹ thuật boosting, giúp tăng cường độ chính xác qua từng vòng lặp bằng cách học từ các sai sót của các cây quyết định trước đó. Điều này giúp XGBoost cải thiện hiệu suất so với mô hình hồi quy tuyến tính, vốn chỉ phù hợp với mối quan hệ tuyến tính giữa các đặc trưng và giá trị dự đoán.

Khác biệt với Random Forest, XGBoost không chỉ tạo ra nhiều cây quyết định độc lập mà còn tập trung vào việc tối ưu hóa mỗi cây dựa trên gradient của hàm mất mát, giúp giảm sai số một cách hiệu quả. Mặc dù Random Forest có thể xử lý các mối quan hệ phi tuyến tính, nhưng XGBoost thường cho kết quả tốt hơn nhờ vào khả năng tối ưu hóa liên tục và điều chỉnh chính xác hơn qua mỗi vòng huấn luyện.

4.4 GridSearchCV

GridSearchCV là một phương pháp được sử dụng để tìm kiếm và tối ưu các siêu tham số (hyperparameters) của một mô hình học máy, nhằm cải thiện độ chính xác và hiệu suất của mô hình. Nó thực hiện thử tất cả các tổ hợp các giá trị siêu tham số có thể có từ một "lưới" các giá trị đã được chỉ định trước.

Nguyên lý hoạt động:

GridSearchCV tìm kiếm trong không gian siêu tham số bằng cách thử nghiệm với mọi giá trị của các tham số đã cho, sử dụng phương pháp k-fold cross-validation để đánh giá hiệu suất của mỗi kết hợp tham số. Mục tiêu là tìm ra bộ tham số có thể tối ưu hóa độ chính xác của mô hình.

Quá trình này có thể được mô tả như sau:

$$\text{Best Parameters} = \arg \max_{\theta} \left(\frac{1}{k} \sum_{i=1}^k \text{score}(X_{train}^{(i)}, y_{train}^{(i)}, \theta) \right)$$

Trong đó:

- θ là bộ tham số của mô hình.
- $X_{train}^{(i)}$ và $y_{train}^{(i)}$ là dữ liệu huấn luyện tại lần gập thứ i trong quá trình k-fold cross-validation.
- $\text{score}(X, y, \theta)$ là độ chính xác của mô hình với tham số θ trên bộ dữ liệu X và y .

Tối ưu siêu tham số với GridSearchCV

Trong bài toán này, nhóm sẽ kết hợp giữa phương pháp bagging và boosting cây quyết định, cụ thể là các mô hình Random Forest và XGBoost, với kỹ thuật GridSearchCV để tối ưu hóa các tham số của mô hình. Phương pháp bagging (Bootstrap Aggregating) được áp dụng với Random Forest, trong khi đó boosting được sử dụng với XGBoost, nhằm tăng cường hiệu quả dự đoán và giảm thiểu lỗi.

Cả hai mô hình Random Forest và XGBoost đều có các siêu tham số quan trọng có thể điều chỉnh để tối ưu hóa hiệu suất. Với GridSearchCV, nhóm sẽ tìm kiếm và chọn lựa các giá trị siêu tham số tối ưu thông qua việc thử nghiệm tất cả các kết hợp có thể của các tham số trong không gian tìm kiếm. Quá trình này giúp mô hình đạt được hiệu quả dự đoán tốt nhất cho bài toán cụ thể.

GridSearchCV sẽ giúp tìm ra các tham số tối ưu của mô hình thông qua việc đánh giá hiệu suất mô hình trên một tập kiểm tra (cross-validation) với các tham số khác nhau, từ đó chọn ra bộ tham số tối ưu giúp mô hình hoạt động hiệu quả nhất. Các tham số sẽ bao gồm:

- **Random Forest:** số lượng cây ($n_estimators$), độ sâu tối đa của cây (max_depth), số lượng mẫu tối thiểu để chia tách ($min_samples_split$).
- **XGBoost:** số lượng cây ($n_estimators$), tốc độ học ($learning_rate$), độ sâu của cây (max_depth), tỷ lệ mẫu ($subsample$).

Việc kết hợp giữa bagging, boosting và GridSearchCV giúp mô hình cải thiện khả năng dự đoán trong các bài toán phức tạp như dự báo giá nhà, từ đó giúp mô hình có khả năng học được các mối quan hệ phi tuyến tính và giảm thiểu overfitting.

4.5 Mạng nơ-ron nhiều lớp

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một mô hình tính toán được lấy cảm hứng từ cấu trúc và chức năng của hệ thần kinh sinh học. Trong đó, mạng nơ-ron nhiều lớp (Multilayer Perceptron - MLP) là một dạng mạng nơ-ron phổ biến và quan trọng, có khả năng học các mối quan hệ phi tuyến giữa đầu vào và đầu ra thông qua nhiều tầng xử lý thông tin. Mạng nơ-ron nhiều lớp là một loại mạng nơ-ron truyền thẳng (feedforward neural network), trong đó các nơ-ron được tổ chức thành từng tầng, bao gồm: tầng đầu vào, một hoặc nhiều tầng ẩn, và tầng đầu ra. Mỗi tầng đều gồm nhiều nơ-ron, trong đó mỗi nơ-ron của một tầng bất kỳ đều được kết nối đầy đủ với tất cả các nơ-ron của tầng liền trước và tầng liền sau, tạo thành một mạng có kết nối dày đặc (fully connected).

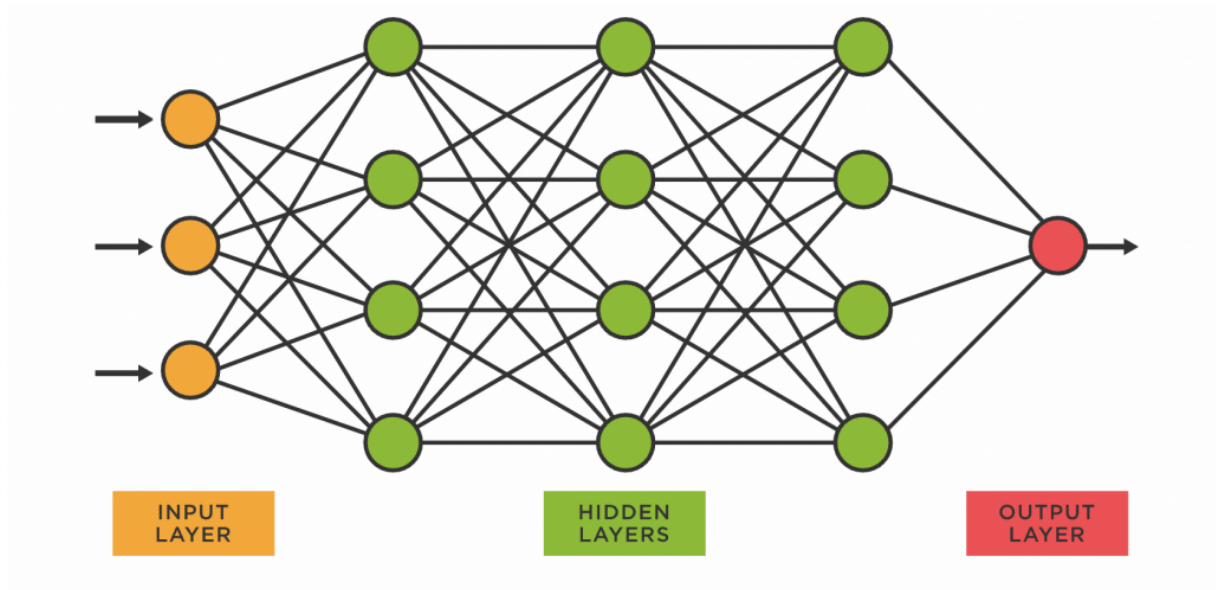


Figure 4.1: Mạng nơ-ron nhân tạo nhiều lớp

Nguyên lý hoạt động:

Quá trình xử lý dữ liệu trong MLP được chia thành hai giai đoạn chính: lan truyền tiến (forward propagation) và lan truyền ngược (backpropagation). Trong giai đoạn lan truyền tiến, dữ liệu đầu vào được truyền từ tầng đầu vào qua các tầng ẩn tới tầng đầu ra. Mỗi nơ-ron thực hiện phép biến đổi tuyến tính với trọng số và hệ số dịch như sau:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \phi(z^{(l)})$$

Trong đó:

- $W^{(l)}$ là ma trận trọng số của tầng l
- $b^{(l)}$ là vector hệ số dịch (bias)
- $a^{(l-1)}$ là đầu ra của tầng trước
- ϕ là hàm kích hoạt phi tuyến

Sau khi đầu ra cuối cùng được tính toán, hàm mất mát (loss function) sẽ đo độ sai lệch giữa đầu ra dự đoán và nhãn thực tế. Hàm mất mát phổ biến trong bài toán phân loại

là hàm cross-entropy, trong khi bài toán hồi quy thường dùng hàm MSE (Mean Squared Error).

Để cập nhật trọng số nhằm giảm sai số, thuật toán lan truyền ngược được sử dụng. Dựa trên đạo hàm của hàm mất mát đối với các trọng số, ta cập nhật từng trọng số theo hướng giảm gradient:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

Trong đó η là tốc độ học (learning rate), quyết định mức độ điều chỉnh sau mỗi lần cập nhật. Việc cập nhật có thể được thực hiện bằng nhiều thuật toán tối ưu khác nhau như Gradient Descent, Stochastic Gradient Descent (SGD), hay Adam — một phương pháp hiện đại và phổ biến vì khả năng hội tụ nhanh và ổn định.

Vai trò của hàm kích hoạt và khả năng biểu diễn phi tuyến

Một điểm then chốt trong kiến trúc của MLP là việc sử dụng các hàm kích hoạt phi tuyến. Nếu toàn bộ mạng chỉ bao gồm các phép biến đổi tuyến tính, thì dù có nhiều tầng cũng không thể biểu diễn được hàm phi tuyến phức tạp. Hàm kích hoạt giúp mạng "bẻ cong" không gian đặc trưng, từ đó học được các mối quan hệ phức tạp trong dữ liệu.

Một số hàm kích hoạt phổ biến như:

- **Sigmoid:** thích hợp cho đầu ra nhị phân nhưng dễ bị "vanishing gradient".
- **Tanh:** cải tiến so với sigmoid do có đầu ra đối xứng quanh 0, nhưng vẫn bị mất gradient ở biên.
- **ReLU:** đơn giản và hiệu quả, giúp giải quyết vấn đề mất gradient, tuy nhiên có thể bị "chết nơ-ron" nếu giá trị đầu vào âm liên tục.
- **Leaky ReLU:** một biến thể của ReLU có thể giải quyết vấn đề chết nơ-ron bằng cách cho phép gradient nhỏ hơn 0 ở phía âm.

Nhờ vào các hàm kích hoạt này, MLP có thể xấp xỉ bất kỳ hàm liên tục nào, theo định lý xấp xỉ phổ quát (Universal Approximation Theorem), nếu được cung cấp đủ nơ-ron và dữ liệu huấn luyện phù hợp.

Trong khi các phương pháp hồi quy tuyến tính hoặc cây quyết định có thể tốt với các mối quan hệ đơn giản hoặc có cấu trúc rõ ràng, MLP có thể học được các mối quan hệ không tuyến tính mà các phương pháp này không thể mô phỏng được. Điều này đặc biệt quan trọng trong dự báo giá nhà, vì giá trị bất động sản thường bị ảnh hưởng bởi nhiều yếu tố phức tạp, chẳng hạn như vị trí, kích thước, tình trạng tài chính của người mua, hoặc các yếu tố kinh tế vĩ mô mà không thể dễ dàng mô hình hóa bằng các phương pháp thông thường.

Với khả năng xấp xỉ các hàm phi tuyến, MLP có thể nắm bắt các mẫu dữ liệu phức tạp và linh hoạt hơn, đặc biệt khi dữ liệu huấn luyện có nhiều đặc trưng và mối quan hệ giữa chúng không phải là tuyến tính. Điểm phá của MLP so với các phương pháp truyền thống là khả năng học từ dữ liệu mà không cần phải đưa ra giả định về mối quan hệ giữa các đặc trưng. MLP có thể tự động tìm ra các mối quan hệ phi tuyến, điều mà các mô hình hồi quy tuyến tính không thể thực hiện, từ đó nâng cao độ chính xác của dự báo giá nhà, đặc biệt trong các trường hợp có sự thay đổi hoặc biến động mạnh trong các yếu tố ảnh hưởng.

5 Xây dựng mô hình dự đoán

Sau khi hoàn tất bước tiền xử lý dữ liệu, tập dữ liệu được chia thành hai phần: tập huấn luyện (training set) và tập kiểm tra (test set) theo tỷ lệ 80:20. Việc phân chia này nhằm đảm bảo mô hình được đánh giá trên dữ liệu chưa từng thấy, từ đó phản ánh chính xác khả năng tổng quát hóa.

```
# =====  
# Train-test split  
# =====  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Figure 5.1: Chia tập dữ liệu Train/Test

Mặc dù các đặc trưng trong tập dữ liệu đều ở dạng số, nhưng chúng có sự khác biệt lớn về thang đo. Điều này có thể ảnh hưởng tiêu cực đến hiệu quả của một số mô hình, đặc

biệt là các mô hình tuyến tính.

Do đó, kỹ thuật chuẩn hóa dữ liệu **StandardScaler** được sử dụng để đưa các đặc trưng về cùng một phân phối với trung bình bằng 0 và độ lệch chuẩn bằng 1. Việc chuẩn hóa giúp cải thiện tốc độ hội tụ và độ ổn định trong quá trình huấn luyện.

```
# =====  
# Standard Scaler  
# =====  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Figure 5.2: Chuẩn hóa dữ liệu với StandardScaler

Sau khi chuẩn bị dữ liệu, các mô hình được huấn luyện và đánh giá như sau:

5.1 Hồi quy tuyến tính

Mô hình hồi quy tuyến tính được huấn luyện trên dữ liệu đã được chuẩn hóa. Đây là mô hình cơ sở (baseline) nhằm thiết lập một mức tham chiếu cho hiệu suất của các mô hình phức tạp hơn.

```
# =====  
# 1. Linear Regression  
# =====  
lr_model = LinearRegression()  
lr_model.fit(X_train_scaled, y_train)  
  
y_pred_lr = lr_model.predict(X_test_scaled)
```

Figure 5.3: Mô hình hồi quy tuyến tính



5.2 Random Forest

Đối với mô hình Random Forest, nhóm sử dụng phương pháp **GridSearchCV** để tìm kiếm bộ siêu tham số tối ưu. Việc này giúp cải thiện hiệu suất mô hình một cách đáng kể.

Các siêu tham số được tinh chỉnh bao gồm:

- **n_estimators**: số lượng cây trong rừng
- **max_depth**: độ sâu tối đa của cây
- **max_features**: số đặc trưng được xét tại mỗi lần phân tách
- **min_samples_split**: số mẫu tối thiểu để chia node
- **min_samples_leaf**: số mẫu tối thiểu tại node lá
- **bootstrap**: sử dụng bootstrap sampling hay không

```
# =====  
# 2. Random Forest + GridSearch  
# =====  
rf = RandomForestRegressor(random_state=42)  
  
param_grid = {  
    "n_estimators": [100, 200],  
    "max_depth": [None, 10, 20],  
    "max_features": ["sqrt", "log2"],  
    "min_samples_split": [2, 5],  
    "min_samples_leaf": [1, 2],  
    "bootstrap": [True]  
}  
  
grid_search = GridSearchCV(  
    rf,  
    param_grid,  
    cv=5,  
    scoring="neg_mean_squared_error",  
    n_jobs=-1  
)  
  
grid_search.fit(X_train, y_train)  
  
best_rf = grid_search.best_estimator_  
  
y_pred_rf = best_rf.predict(X_test)
```

Figure 5.4: Tối ưu tham số với GridSearchCV

Sau quá trình tìm kiếm, mô hình thu được bộ tham số tối ưu giúp đạt hiệu suất tốt nhất trên tập validation.

```
Bộ tham số tốt nhất:  
{'colsample_bytree': 0.8, 'gamma': 0.1, 'learning_rate': 0.1, 'max_depth': 8, 'n_estimators': 300, 'subsample': 0.6}
```

Figure 5.5: Bộ tham số tối ưu

5.3 XGBoost

Mô hình XGBoost được xây dựng và tối ưu hóa siêu tham số thông qua thuật toán tìm kiếm dạng lưới **GridSearchCV**. Quá trình này được kết hợp với phương pháp kiểm định chéo và sử dụng thang đo sai số toàn phương trung bình (Mean Squared Error - MSE) làm hàm mục tiêu đánh giá.

Không gian tìm kiếm được thiết lập với các tham số cụ thể như sau:

- **n_estimators** (Tập giá trị thử nghiệm: [100, 200, 300]): Số lượng cây quyết định được xây dựng. Mỗi cây mới sẽ học cách bù đắp sai số cho các cây trước đó.
- **learning_rate** (Tập giá trị thử nghiệm: [0.01, 0.1]): Điều chỉnh mức độ ảnh hưởng của mỗi cây mới lên mô hình cuối cùng. Giá trị nhỏ giúp mô hình hội tụ ổn định hơn và giảm thiểu rủi ro quá khớp (overfitting).
- **max_depth** (Tập giá trị thử nghiệm: [6, 8, 10]): Chiều sâu tối đa của mỗi cây. Cây càng sâu càng nắm bắt được các quan hệ phi tuyến phức tạp, nhưng đồng thời làm tăng nguy cơ overfitting.
- **subsample** (Tập giá trị thử nghiệm: [0.6, 0.8, 1.0]): Tỷ lệ lấy mẫu ngẫu nhiên từ tập dữ liệu huấn luyện để xây dựng từng cây. Đặt giá trị nhỏ hơn 1 đóng vai trò như một kỹ thuật regularization.
- **colsample_bytree** (Tập giá trị thử nghiệm: [0.7, 0.8, 1.0]): Tỷ lệ đặc trưng (features) được chọn ngẫu nhiên để huấn luyện tại mỗi cây, giúp các cây trong mô hình đa dạng hơn.
- **gamma** (Tập giá trị thử nghiệm: [0.1, 0.3, 0.5]): Ngưỡng giảm hàm mất mát tối thiểu yêu cầu để một node tiếp tục được phân nhánh. Đây là tham số quan trọng để kiểm soát độ phức tạp của cấu trúc cây.

```
def train_xgboost_model(X_train: np.ndarray, y_train: pd.Series, isBest: bool = True) -> GridSearchCV:
    xgbr = XGBRegressor(random_state=42)

    if not isBest:
        params = {
            'n_estimators': [100, 200, 300],
            'learning_rate': [0.01, 0.1],
            'subsample': [0.6, 0.8, 1.0],
            'colsample_bytree': [0.7, 0.8, 1.0],
            'max_depth': [6, 8, 10],
            'gamma': [0.1, 0.3, 0.5],
        }
    else:
        params = {
            'n_estimators': [300],
            'learning_rate': [0.1],
            'subsample': [0.6],
            'colsample_bytree': [0.8],
            'max_depth': [8],
            'gamma': [0.1],
        }

    grid_search = GridSearchCV(
        estimator=xgbr, param_grid=params, cv=5,
        scoring='neg_mean_squared_error', return_train_score=True, n_jobs=-1
    )
    grid_search.fit(X_train, y_train)
    return grid_search
```

Figure 5.6: Cấu hình không gian tìm kiếm GridSearchCV cho XGBoost

Sau khi rà soát toàn bộ các tổ hợp trong không gian siêu tham số, thuật toán đã hội tụ và xác định được bộ tham số tối ưu nhất, giúp mô hình đạt sai số thấp nhất trên tập kiểm định chéo. Kết quả bộ tham số tốt nhất được thể hiện như sau:

```
Bộ tham số tốt nhất:
{'colsample_bytree': 0.8, 'gamma': 0.1, 'learning_rate': 0.1, 'max_depth': 8, 'n_estimators': 300, 'subsample': 0.6}
```

Figure 5.7: Bộ tham số tốt nhất cho mô hình

5.4 Mạng Nơ-ron nhiều lớp

Ta tiến hành sử dụng phương pháp Nơ-ron nhiều lớp với cấu trúc như trong hình 5.8.

```
class NeuralNetMLP(nn.Module):
    def __init__(self, in_features=12):
        super(NeuralNetMLP, self).__init__()
        self.layer_1 = nn.Linear(in_features, 64)
        self.act_1 = nn.ReLU()
        self.layer_2 = nn.Linear(64, 128)
        self.act_2 = nn.ReLU()
        self.output_layer = nn.Linear(128, 1)

    def forward(self, input_data):
        out = self.layer_1(input_data)
        out = self.act_1(out)
        out = self.layer_2(out)
        out = self.act_2(out)
        out = self.output_layer(out)
        return out
```

Figure 5.8: Phương pháp mạng Nơ-ron nhiều lớp

Ta tiến hành huấn luyện mô hình với các tham số huấn luyện:

- Learning rate: 0.001
- Hàm mất mát: MSE
- Hàm tối ưu: Adam
- Số epoch: 35

Sau khi huấn luyện, ta tiến hành vẽ đồ thị mất mát. Thông qua đồ thị cho thấy cả Train Loss và Validation Loss đều giảm dần theo số epoch, với Train Loss giảm nhanh hơn trong 10 epoch đầu, sau đó cả hai đường ổn định từ epoch 20. Validation Loss luôn cao hơn Train Loss nhưng khoảng cách không lớn, chứng tỏ mô hình không bị overfitting nghiêm trọng. Đến khoảng 30-35 epoch, cả hai đường gần như không giảm thêm, đạt điểm hội tụ.

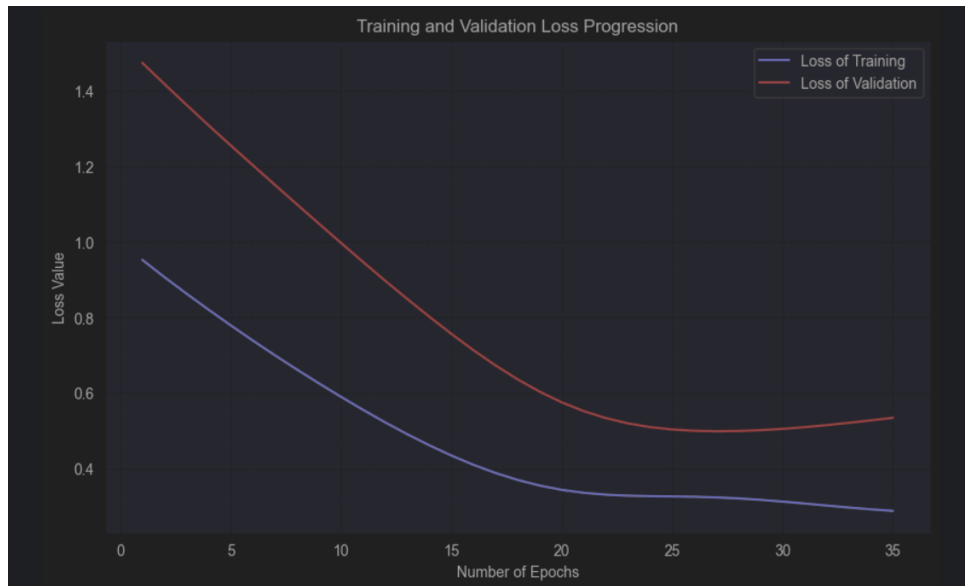


Figure 5.9: Đồ thị mất mát

6 Đánh giá mô hình dự đoán

6.1 Các Phương pháp Đánh giá Hiệu quả Mô hình

Để đánh giá hiệu quả của các mô hình học máy, bốn chỉ số phổ biến được sử dụng như sau:

- RMSE (Root Mean Square Error):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- RRMSE (Relative RMSE):

$$RRMSE = \frac{RMSE}{\bar{y}}$$

- R-squared (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- **MAE (Mean Absolute Error):**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

6.2 Kết quả đánh giá

Sau khi huấn luyện, nhóm thu được các chỉ số đánh giá như sau:

Table 6.1: Các chỉ số đánh giá hiệu suất mô hình

Phương Pháp	RMSE	RRMSE	R^2	MAE
Linear Regression	1,324,506.96	0.2645	0.653	970,043.40
Random Forest	1,400,619.50	0.2797	0.612	1,010,525.30
XG_Boost	1,352,015.72	0.27	0.64	976,624.56
Mạng Nơ-ron	1,347,227.33	0.27	0.64	956,810.86

Dựa trên kết quả thu được, có thể rút ra các nhận xét chi tiết sau:

- Mô hình **Hồi quy tuyến tính (Linear Regression)** đạt hiệu suất tổng thể tốt nhất dựa trên chỉ số RMSE thấp nhất (1,324,506.96) và hệ số xác định R^2 cao nhất (0.653). Điều này cho thấy tập dữ liệu có xu hướng phân bố tuyến tính khá mạnh, giúp một phương pháp cơ bản phát huy tối đa sức mạnh mà không gặp tình trạng quá khớp (overfitting).
- **Mạng Nơ-ron (MLP)** và **XGBoost** bám sát ngay sau với mức hiệu năng vô cùng ổn định (R^2 đều đạt mức 0.64). Đáng chú ý, Mạng Nơ-ron lại là mô hình sở hữu sai số tuyệt đối trung bình (MAE) thấp nhất (956,810.86). Điều này chứng tỏ MLP có khả năng dự đoán các mức giá nhà thông thường rất sát với thực tế, dù thỉnh thoảng nhạy cảm với các điểm dị biệt (outliers) khiến chỉ số RMSE bị đẩy lên cao hơn so với Hồi quy tuyến tính.
- Mô hình **Random Forest** thể hiện hiệu suất kém nhất trong cả bốn phương pháp. Mặc dù là một thuật toán mạnh về khả năng mô hình hóa quan hệ phi tuyến, nhưng đối với tập dữ liệu này, thuật toán cây phân quyết định không mang lại hiệu quả, dẫn đến các chỉ số sai số cao nhất.

Kết luận: Dữ liệu trong bài toán mang đậm tính chất tuyến tính, do đó mô hình Hồi quy tuyến tính lại trở thành lựa chọn an toàn và hiệu quả nhất về mặt tổng thể. Tuy nhiên, nếu ưu tiên độ chính xác trên trung bình (dựa theo MAE), Mạng Nơ-ron nhiều lớp hoàn toàn là một giải pháp dự đoán đáng tin cậy.

6.3 Đồ thị đánh giá

Để trực quan hóa hiệu quả mô hình, nhóm sử dụng ba loại biểu đồ:

- **Predictions vs Actual:** đánh giá độ chính xác tổng thể
- **Residuals vs Predicted:** kiểm tra lỗi hệ thống
- **Distribution of Residuals:** đánh giá phân phối sai số

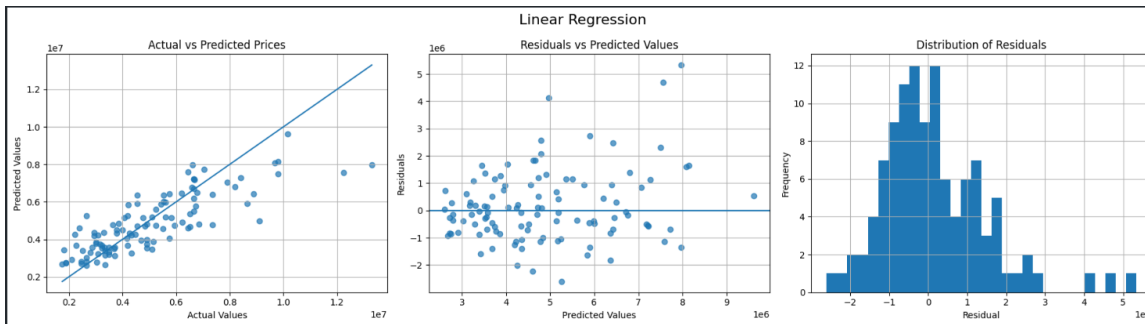


Figure 6.1: Linear Regression: (a) Predictions vs Actual, (b) Residuals vs Predictions, (c) Distribution of Residuals

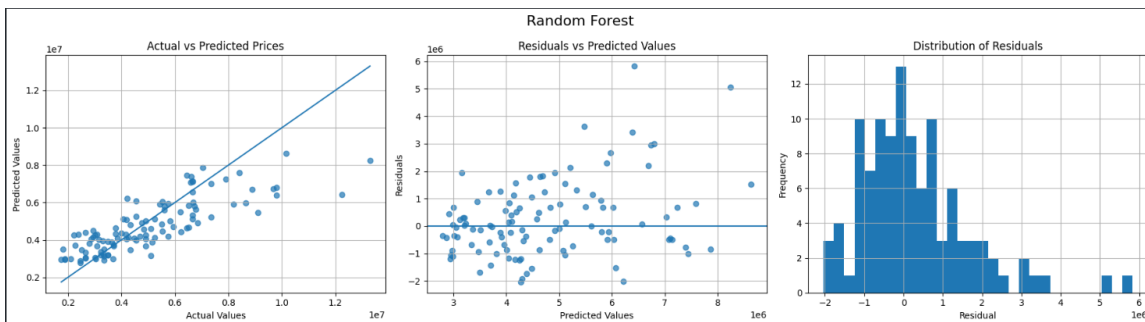


Figure 6.2: Random Forest: (a) Predictions vs Actual, (b) Residuals vs Predictions, (c) Distribution of Residuals

Figure 6.3: XGBoost

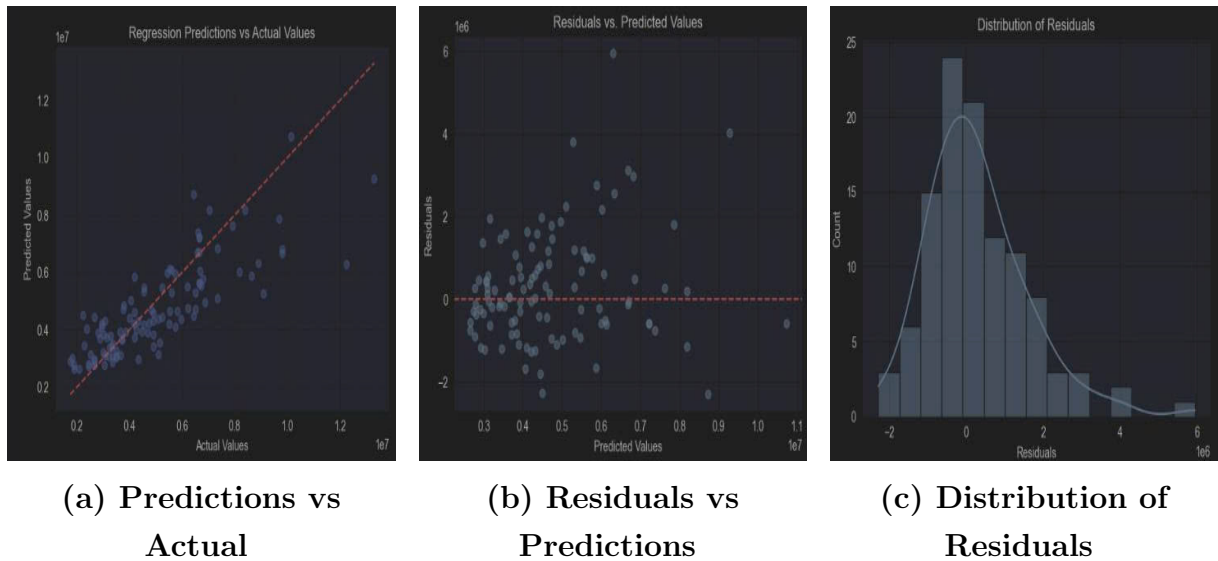
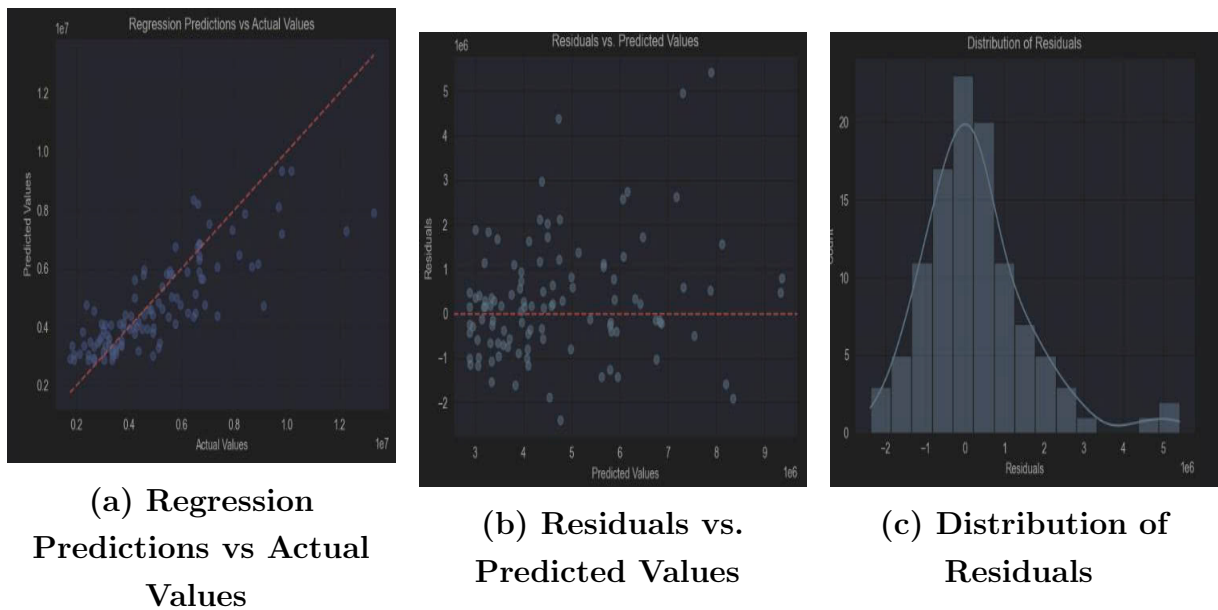


Figure 6.4: Mạng Nơ-ron nhiều lớp



Từ các biểu đồ phân tích và giá trị dự đoán, có thể nhận thấy:

- Với **Linear Regression**, các điểm dữ liệu phân bố khá sát đường lý tưởng $y = \hat{y}$. Phần dư phân bố ngẫu nhiên đều quanh trục 0 và biểu đồ phân phối phần dư có dạng hình quả chuông gần với phân phối chuẩn nhất, cho thấy mô hình dự đoán rất đồng đều và ổn định.

- Với **Random Forest**, độ phân tán của các điểm lớn hơn rõ rệt, đặc biệt phần dư có xu hướng loe rộng ra ở phân khúc các căn nhà có giá trị cao. Phân phối phần dư có xu hướng dẹt hơn, minh chứng cho việc mô hình gặp khó khăn và sai số lớn hơn so với mô hình tuyến tính.
- Với **XGBoost**, biểu đồ cho thấy sự cải thiện đáng kể so với Random Forest khi các điểm dự đoán bám sát đường lý tưởng hơn. Tuy nhiên, ở vùng giá trị cực đại, phần dư vẫn xuất hiện sự phân tán nhất định, chưa duy trì được độ hội tụ xuyên suốt như Linear Regression.
- Với **Mạng Nơ-ron (MLP)**, mật độ các điểm dữ liệu tập trung cực kỳ dày đặc và bám rất sát đường lý tưởng ở các phân khúc giá trung bình và thấp (phản ánh đúng việc mô hình đạt chỉ số MAE thấp nhất). Tuy nhiên, đồ thị phần dư cũng bộc lộ rõ một vài điểm dị biệt nằm rải rác khá xa trục 0, giải thích nguyên nhân chỉ số RMSE bị phạt nặng và cao hơn so với Hồi quy tuyến tính.

6.4 Kết luận

Thông qua quá trình huấn luyện, tối ưu hóa siêu tham số và đánh giá trực quan, nhóm có thể kết luận rằng **Hồi quy tuyến tính (Linear Regression)** là lựa chọn phù hợp và tối ưu nhất cho bài toán dự đoán giá nhà trên tập dữ liệu này. Nguyên nhân cốt lõi là do dữ liệu đầu vào thể hiện xu hướng tuyến tính rõ rệt, giúp một phương pháp thống kê cơ bản phát huy tối đa hiệu năng (RMSE thấp nhất, R^2 cao nhất), đồng thời tiết kiệm tuyệt đối chi phí tính toán.

Bên cạnh đó, **Mạng Nơ-ron nhiều lớp (MLP)** nổi lên như một giải pháp cực kỳ tiềm năng nếu hệ thống triển khai thực tế ưu tiên sai số tuyệt đối thấp nhất cho các căn nhà phổ thông (chỉ số MAE tốt nhất).

Ngược lại, các mô hình học máy theo dạng tập hợp phức tạp như Random Forest hay XGBoost lại không mang lại sự đột phá đáng kể về mặt độ chính xác, thậm chí tiêu tốn nhiều tài nguyên hơn cho việc tìm kiếm siêu tham số, do đó không phải là hướng tiếp cận hiệu quả nhất trong bối cảnh dữ liệu hiện tại.

7 Tổng Kết

Trong bài tập lớn lần này, nhóm đã tiến hành đầy đủ các bước trong quy trình khai phá tri thức nhằm tìm ra mô hình phù hợp nhất để giải quyết bài toán dự đoán giá nhà, sử dụng tập dữ liệu gồm 545 ngôi nhà với các thuộc tính đặc trưng có giá trị cho quá trình huấn luyện.

Từ dữ liệu thô thu thập được trên nền tảng Kaggle, nhóm đã lần lượt thực hiện các bước: làm sạch dữ liệu, tích hợp dữ liệu, chọn lọc thuộc tính, chuyển đổi dữ liệu, khai phá dữ liệu, đánh giá mô hình và biểu diễn tri thức. Bốn phương pháp khai phá dữ liệu được áp dụng gồm: Hồi quy tuyến tính, Random Forest, XGBoost và mạng Nơ-ron nhiều lớp (Neural Network), nhằm so sánh và lựa chọn mô hình tối ưu.

Thông qua quá trình huấn luyện và đánh giá, cả bốn mô hình đều cho kết quả khả quan. Tuy nhiên, mô hình **Hồi quy tuyến tính (Linear Regression)** thể hiện hiệu năng tổng thể vượt trội nhất (đạt chỉ số RMSE thấp nhất và R^2 cao nhất). Điều này xuất phát từ việc các đặc trưng trong tập dữ liệu mang đậm tính chất tuyến tính, giúp một mô hình thống kê cơ bản phát huy tối đa sức mạnh, đảm bảo khả năng tổng quát hóa cao mà không bị rơi vào trạng thái quá khớp (overfitting).

Bên cạnh đó, **Mạng Nơ-ron nhiều lớp (MLP)** cũng là một điểm sáng đáng ghi nhận khi đạt mức sai số tuyệt đối trung bình (MAE) thấp nhất trong cả bốn phương pháp, cho thấy khả năng dự đoán cực kỳ sát sao đối với các phân khúc nhà phổ thông. Mô hình **XGBoost** xếp ngay sau với sự hội tụ ổn định và độ tin cậy cao.

Tóm lại, để triển khai trong hệ thống dự đoán giá nhà thực tế, Hồi quy tuyến tính là lựa chọn ưu tiên hàng đầu nhờ sự cân bằng hoàn hảo giữa độ chính xác, chi phí tài nguyên tính toán thấp và tính dễ diễn giải. Nếu hệ thống đặc thù yêu cầu tối thiểu hóa sai số tuyệt đối trên từng căn nhà cụ thể, mạng Nơ-ron sẽ là một phương án triển khai xuất sắc.